Rapport de projet ALGO

Achraf,Thomas,Elouen,Woody

Janvier 2024

Contents

1 Introduction

- Objectif 1 :

2 Analyses

2.1 Compression

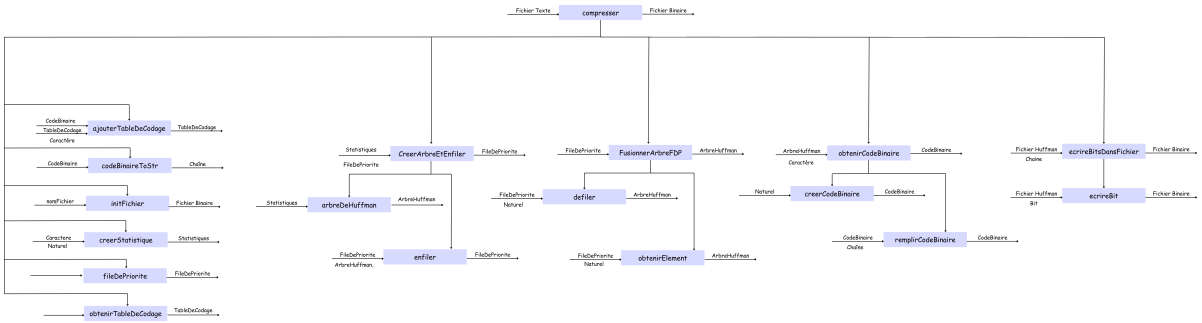


Figure 1: Analyse Descendante Compression

2.2 Decompression

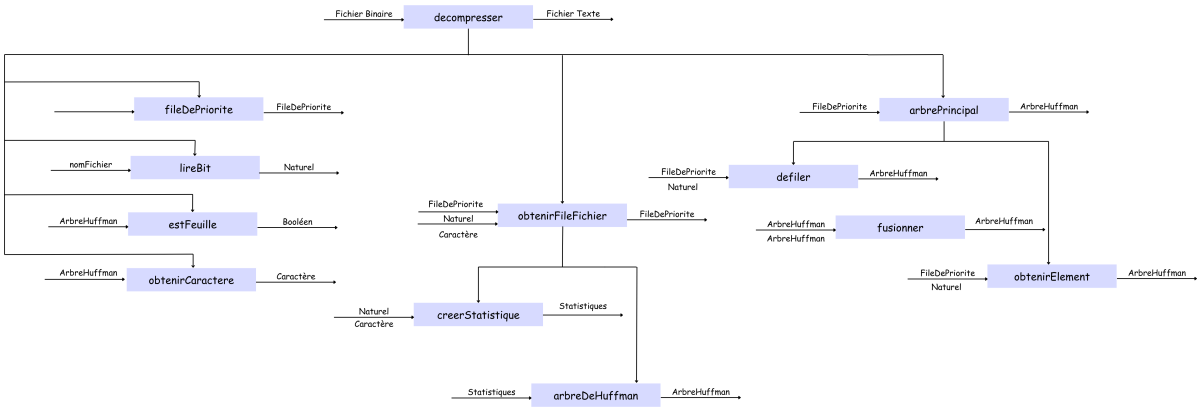


Figure 2: Analyse Descendante Décompression

2.3 TAD Code Binaire

Nom: Code Binaire

Utilise: TableDeCodage, Caractere, Bit

Opérations: creerCodeBinaire: **Naturel** $\rightarrow$ CodeBinaire

remplirCodeBinaire: **Chaine de caracteres** $\times$ CodeBinaire $\nrightarrow$ CodeBinaire

	libererCodeBinaire:	CodeBinaire $\rightarrow$ CodeBinaire
	estVide:	CodeBinaire $\rightarrow$ **Booleen**
	obtenirIemeBit:	CodeBinaire $\times$ **NaturelNonNul** $\nrightarrow$ Bit
	longueur:	CodeBinaire $\rightarrow$ **Naturel**
	concatener:	CodeBinaire $\times$ CodeBinaire $\rightarrow$ CodeBinaire
	codeBinaireToStr:	CodeBinaire $\rightarrow$ **Chaine de caracteres**
	obtenirCodeBinaire:	ArbreHuffman $\times$ **Caractere** $\nrightarrow$ **Chaine de caracteres**
	obtenirCodeBinaireR:	ArbreHuffman $\times$ **Caractere** $\nrightarrow$ **Chaine de caracteres**
	CbFromAH:	ArbreHuffman $\times$ **Caractere** $\rightarrow$ CodeBinaire
	codeBinaireToStr:	CodeBinaire $\rightarrow$ **Chaine de caracteres**
Sémantiques:	creerCodeBinaire:	Fonction qui creer un code binaire à partir de sa taille
	remplirCodeBinaire:	Fonction qui permet de remplir un code binaire avec une chaîne de caractère
	libererCodeBinaire:	Fonction qui libère un code binaire
	estVide:	Fonction qui permet de savoir si un code est vide ou non
	obtenirIemeBit:	Fonction qui permet d'obtenir le ieme bit d'un code binaire
	concatener:	Fonction qui permet de concatener 2 codes binaires
	codeBinaireToStr:	Fonction qui permet de convertir un code binaire en chaîne de caractères
	obtenirCodeBinaire:	Fonction qui va permettre d'avoir le code binaire d'un caractere
	obtenirCodeBinaireR:	Fonction récursive qui va permettre d'avoir le code binaire d'un caractere et qui va être appelé dans obtenirCodeBinaire
	CbFromAH:	Fonction qui donne un code binaire à partir d'un arbre d'Huffman et d'un caractere
	codeBinaireToStr:	Fonction qui transforme un code binaire en chaine de caracteres
Préconditions:	remplirCodeBinaire(code):	$longueur(chaine) \neq code.taille$
	obtenirIemeBit(code, i):	$i > 0$ et $i \leq longueur(code)$
	obtenirCodeBinaire(a, car):	non ArbreHuffman.estVide(a)
	obtenirCodeBinaireR(a, car):	non ArbreHuffman.estVide(a)

2.4 TAD Octet

Nom: Octet

Utilise: Chaine de caracteres, Code Binaire, Booleen, Bit

Opérations:

creerOctet:	$\rightarrow$ Octet
fixerBit:	Octet **Entier**Bit $\rightarrow$ Octet
iemeBit:	Octet **Entier** $\rightarrow$ Bit
decaler:	Octet $\rightarrow$ Octet
bitsToStr:	Octet $\rightarrow$ **Chaine de caracteres**
strToBits:	**Chaine de caracteres** $\rightarrow$ Octet
encoder:	Octet $\times$ Code Binaire $\nrightarrow$ Octet
estVide:	Octet $\rightarrow$ **Booleen**
inverser:	Octet $\rightarrow$ Octet

Sémantiques:

creerOctet:	Permet d'obtenir un octet avec tous ses bits à 0
fixerBit:	Permet de fixer les bits d'un octet
iemeBit:	Renvoie le i-eme bit d'un octet
decaler:	Decale l'octet vers la gauche
bitsToStr:	Renvoie un octet en chaîne de caractere
strToBits:	Renvoie l'octet d'une chaîne de caractere
encoder:	Remplit un octet avec un code binaire
estVide:	Verfie si l'octet est vide
inverser:	Inverse l'octet

Préconditions: encoder(o, code,i): non estVide(code), $0 \leq i \leq 7$

2.5 TAD Statistiques

Nom: Statistiques

Utilise: Caractère, Chaine de caracteres, Naturel

Opérations:

statistiques: **Caractere** $\times$ **Naturel** $\rightarrow$ Statistiques

obtenirOccurences: Statistiques $\rightarrow$ **Naturel**

obtenirCaractere: Statistiques $\rightarrow$ **Caractere**

Sémantiques:

statistiques: Créer une statistique à partir d'un caractère et d'un naturel

obtenirOccurences: Permet d'obtenir le nombre d'occurences d'une statistique

obtenirCaractere: Permet d'obtenir le caractère d'une statistique

2.6 TAD ArbreHuffman

Nom: ArbreHuffman

Utilise: statistiques,**Booleen**

Opérations:

arbreDeHuffman:	Statistiques → ArbreHuffman
fixerStatistique:	ArbreHuffman × Statistiques → ArbreHuffman
obtenirStat:	ArbreHuffman → Statistiques
obtenirCaractere:	ArbreHuffman → **Caractere**
fusionner:	ArbreHuffman × ArbreHuffman ↛ ArbreHuffman
obtenirPonderation:	ArbreHuffman → **Naturel**
fixerFilsDroit:	ArbreHuffman × Arebre Huffman ↛ ArbreHuffman
fixerFilsGauche:	ArbreHuffman × Arebre Huffman ↛ ArbreHuffman
obtenirFilsGauche:	ArbreHuffman ↛ ArbreHuffman
obtenirFilsDroit:	ArbreHuffman ↛ ArbreHuffman
estFeuille:	ArbreHuffman → **Booleen**
estVide:	ArbreHuffman → **Booleen**
estPresent:	ArbreHuffman → **Booleen**
copierArbreHuffman:	ArbreHuffman ↛ ArbreHuffman

Sémantiques:

arbreDeHuffman:	Crée un arbre de huffman à partir d'une statistique
fixerStatistique:	Fixe une statistique à un arbre
obtenirStat:	Retourne la valeur de la statistique d'un arbre
obtenirCaractere:	Retourne le caractère associé à un arbre
fusionner:	Fusionne deux arbres entre eux
obtenirPonderation:	Permet d'obtenir la ponderation d'un arbre de huffman
fixerFilsDroit:	Affecte la valeur au fils droit de l'arbre
fixerFilsGauche:	Affecte la valeur au fils gauche de l'arbre
obtenirFilsGauche:	Permet d'obtenir le fils gauche d'un arbre de huffman

	obtenirFilsDroit:	Permet d'obtenir le fils droit d'un arbre de huffman
	estFeuille:	Verifie si l'arbre de huffman est une feuille
	estVide:	Verifie si l'arbre est vide, si son champ caractère est vide
	estPresent:	Vérifie si un élément est présent dans un arbre
	copierArbreHuffman:	Fonction qui permet de copier un arbre
Préconditions:	fusionner(A):	estVide(A)
	fixerFilsDroit(r,ad):	non(estVide(r))
	fixerFilsGauche(r,ag):	non(estVide(r))
	obtenirFilsGauche(A):	non estUneFeuille(A)
	obtenirFilsDroit(A):	non estUneFeuille(A)
	copierArbreHuffman(arbre):	arbre $\neq$ NULL

2.7 FileDePriorité

Nom:	FileDePriorité	
Utilise:	Booleen	
Opérations:	fileDePriorite:	$\rightarrow$ FileDePriorite
	enfiler:	FileDePriorite $\times$ Element $\rightarrow$ FileDePriorite
	defiler:	FileDePriorite $\rightarrow$ FileDePriorite
	longueur:	FiledePriorite $\rightarrow$ **Naturel**
	estVide:	FileDePriorite $\rightarrow$ **Booleen**
	estPresent:	FileDePriorite $\times$ Element $\rightarrow$ **Booleen**
	obtenirElement:	FileDePriorite $\rightarrow$ Element
Sémantiques:	filedePriorite:	Créer une file de priotité vide
	enfiler:	Enfile un élément en fin de file
	defiler:	Défile le dernier élément de la file
	longueur:	Calcule la longueur de la file
	estVide:	Cherche si la file de priotité est vide ou non
	estPresent:	Cherche si un élément donné est présent dans la file ou non
	obtenirElement:	Donne le dernier élément de la file

2.8 TAD TableDeCodage

Nom:	TableDeCodage	
Utilise:	ArbreHuffmann	
Opérations:	obtenirTableDeCodage:	ArbreHuffman → TableDeCodage
	obtenirCode:	TableDeCodage × **Caractere** ⇸ CodeBinaire
	obtenirCarac:	TableDeCodage × CodeBinaire → **Caractere**
	ajouter:	TableDeCodage × caractere × CodeBinaire → TableDeCodage
	retirer:	TableDeCodage × **Caractere** → TableDeCodage
	estPresent:	TableDeCodage × **Caractere** → **Booleen**
	estVide:	TableDeCodage → **Booleen**
Sémantiques:	obtenirTableDeCodage:	Renvoie la table de codage grâce à un arbre de Huffman
	obtenirCode:	Permet d'obtenir le code binaire d'un caractère grâce à la TableDeCodage
	obtenirCarac:	Permet d'obtenir le caractere d'un code binaire grâce à la TableDeCodage
	ajouter:	Permet d'ajouter une nouveeau caractère avec son code binaire dans la TableDeCodage
	supprimerer:	Permet de supprimer un caractère de la TableDeCodage
	estPresent:	Permet de savoir si un caractère est présent dans la TableDeCodage
	estVide:	Permet de savoir si la TableDeCodage est vide
Préconditions:	obtenirTableDeCodage(A):	non estVide(A)
	obtenirCode(t,c):	estPresent(t,c)

3 Conception préliminaire

3.1 Arbre Huffman

fonction arbreDeHuffman (stat : Statistiques) **:** ˆArbreHuffman

procédure fixerStatistique **(E/S** arbre : ˆArbreHuffman **E** stat : Statistiques)

fonction obtenirStat (arbre : ArbreHuffman) **:** Statistiques

fonction obtenirPonderation (arbre : ArbreHuffman) **: Naturel**

fonction obtenirCaractere (arbre : ArbreHuffman) **: Caractere**

fonction fusionner (arbre1, arbre2 : ˆArbreHuffman) **:** ˆArbreHuffman
⌊**précondition(s)** non estVide(arbre1) ou non estVide(arbre2)

procédure fixerFilsGauche **(E/S** arbre : ˆArbreHuffman, **E** gauche : ˆArbreHuffman)

procédure fixerFilsDroit **(E/S** arbre : ˆArbreHuffman, **E** droit : ˆArbreHuffman)

fonction estFeuille (arbre : Arbre Huffman) **: Booleen**

fonction estVide (arbre: ˆArbreHuffman) **: Booleen**

fonction obtenirFilsGauche (arbre : ˆArbreHuffman) **:** ˆArbreHuffman
⌊**précondition(s)** non estFeuille(arbre)

fonction obtenirFilsDroit (arbre : ˆArbreHuffman) **:** ˆArbreHuffman
⌊**précondition(s)** non estFeuille(arbre)

fonction estPresent (arbre : ˆArbreHuffman, caractere : **Caractere) : Booleen**

fonction copierArbreHuffman (arbre : ˆArbreHuffman) **:** ArbreHuffman
⌊**précondition(s)** arbre $\neq$ NULL

3.2 Code Binaire

fonction creerCodeBinaire (taille : **Naturel) :** CodeBinaire

procédure remplirCodeBinaire **(E/S** code : CodeBinaire, **E** chaine : **Chaine de caracteres)**
⌊**précondition(s)** $longueur(chaine) \neq code.taille$

procédure libererCodeBinaire **(E/S** code : CodeBinaire)

fonction estVide (code : CodeBinaire) **: Booleen**

fonction obtenirIemeBit (code : CodeBinaire, i : **NaturelNonNul) :** Bit
⌊**précondition(s)** $i > 0$ et $i \leq longueur(code)$

fonction concatener (code1, code2: CodeBinaire) **:** CodeBinaire

fonction codeBinaireAStr (code: CodeBinaire) **: Chaine de caracteres**

fonction obtenirCodeBinaire (a : ArbreHuffman, car : **Caractere) : Chaine de caracteres**
⌊**précondition(s)** non ArbreHuffman.estVide(a)

fonction obtenirCodeBinaireR (a : ArbreHuffman, car : **Caractere**) **: Chaine de caracteres**
⌊**précondition(s)** non ArbreHuffman.estVide(a)
fonction CbDeAH (a : ArbreHuffman, car : **Caractere**) **:** CodeBinaire

3.3 Statistiques

fonction statistique (c : **Caractere**, n : **Naturel**) **:** Statistiques
fonction obtenirPonderation (stat : Statistiques) **: Naturel**
fonction obtenirCaractere (stat : Statistiques) **: Caractere**

3.4 Octet

fonction obtenirBitRestants () **:**
fonction obtenirIndicePlusImportantDeBitRestant () **: Entier**
fonction incrementPositionDeDebut () **:**
fonction obtenirStartPosition () **: Entier**
fonction creerOctet () **:** Octet
procédure fixerBit (**E/S** octet : ˆOctet**E** pos : **Entier**, valeur : Bit)
fonction iemeBit (octet : Octet, i : **Entier**) **:** Bit
procédure decaler (**E/S** octet : ˆOctet)
fonction bitsAStr (octet : ˆOctet) **: Chaine de caracteres**
fonction strABits (chaine : **Chaine de caracteres**) **:** Octet
procédure encoder (octet : Octet,cb : CodeBinaire)
⌊**précondition(s)** Octet
fonction estVide (**Booleen**) **:** octet : Octet→
fonction inverser (Octet) **:** octet : Octet

3.5 FileDePriorite

fonction filedePriorite () **:** FileDePriorite
procédure enfiler (**E/S** file : FileDePriorite , **E** arbre : ArbreHuffman)
fonction creerNoeud (arbre : ArbreHuffman) **:** Noeud
fonction trouverIndicePourInsere (arbre : ArbreHuffman, file : FileDePriorite) **: Naturel**
procédure defiler (**E/S** file : FileDePriorite)
⌊**précondition(s)** non(estVide(file))
fonction obtenirElement (file : FileDePriorite, i :**NaturelNonNul**) **:** Arbre-Huffman
⌊**précondition(s)** i<file.longueur

3.6 TableDeCodage

fonction obtenirTableDeCodage (a : ArbreHuffman) **:** TableDeCodage
⌊**précondition(s)** non estVide(a)
fonction obtenirCode (tab : TableDeCodage, c : **Caractere) :** CodeBinaire
⌊**précondition(s)** estPresent(tab,c)
procédure ajouter **(E/S** tab : TableDeCodage **E** c : **Caractere**, code : CodeBinaire)
fonction estPresent (tab : TableDeCodage, c : **Caractere) : Booleen**
fonction estVide (tab : TableDeCodage) **: Booleen**

3.7 Compression

procédure ecrireBit **(E/S** fichier : Fichier Binaire**E** bit : **Caractere)**
procédure ecrireBitsDansFichier **(E/S** fichier : Fichier Binaire**E** bits : **Chaine de caracteres)**
fonction initFichier (nomFichier : **Chaine de caracteres) :** Fichier Texte
procédure CreerArbreEtEnfiler **(E/S** fp : FileDePriorite**E** stat : Statistiques)

fonction FusionnerArbreFDP (fp : FileDePriorite) **:** ArbreHuffman
fonction compresser (nomFichier : **Chaine de caracteres) :** Fichier Binaire

3.8 Decompression

fonction obtenirFileFichier (fp : FileDePriorite, caracter : **Caractere**, ponderation : **Naturel) :** FileDePriorite
fonction lireBit (fichier : Fichier Binaire) **: Naturel**
fonction arbrePrincipal (fp : FileDePriorite) **:** ArbreHuffman
fonction decompresser (nomFichier : Fichier Binaire) **:** Fichier Texte

4 Conception détaillée

4.1 Arbre Huffman

4.1.1 Déclaration du type

Type ArbreHuffman = ^Racine
Type Racine = **Structure**
filsGauche : ArbreHuffman
filsDroit : ArbreHuffman
statistique : Statistiques
finstructure

4.1.2 arbrehuffman

fonction arbrehuffman (stat : Statistiques) **:** ArbreHuffman
Déclaration arbre : ArbreHuffman
debut
arbre.filsGauche ← NIL
arbre.filsDroit ← NIL
arbre.statistique ← stat
retourner arbre
fin

4.1.3 fusionner

fonction fusionner (arbre1 , arbre2 : ArbreHuffman) **:** ArbreHuffman
⌊**précondition(s)** non (estVide(arbre1) et estVide(arbre2))
Déclaration arbre : ArbreHuffman
debut
arbre ← arbrehuffman()
arbre.statistique.caractere ← NULL
arbre.statistique.ponderation ← obtenirPonderation(arbre1) + obtenirPonderation(arbre2)
si obtenirPonderation(arbre1)<obtenirPonderation(arbre2) **alors**
arbre.filsGauche ← arbre1
arbre.filsDroit ← arbre2
sinon
arbre.filsDroit ← arbre1
arbre.filsGauche ← arbre2
finsi
retourner arbre
fin

4.1.4 obtenirPonderation

fonction obtenirPonderation (arbre : ArbreHuffman) **: Naturel**

debut
  retourner arbre.statistique.ponderation
fin

4.1.5 fixerFilsDroit

procédure fixerFilsDroit **(E/S** arbre : ArbreHuffman **E** droit : ArbreHuffman**)**
debut
  arbre.filsDroit ← droit
fin

4.1.6 fixerFilsGauche

procédure fixerFilsGauche **(E/S** arbre : ArbreHuffman **E** gauche : ArbreHuffman**)**
debut
  arbre.filsGauche ← gauche
fin

4.1.7 estVide

fonction estVide **(**arbre : ArbreHuffman**) : Booleen**
debut
  retourner ((arbre.statistique.caractere = NULL) et (arbre.statistique.ponderation = 0))
fin

4.1.8 estFeuille

fonction estFeuille **(**arbre : ArbreHuffman**) : Booleen**
debut
  retourner (estVide(arbre.filsDroit) et estVide(arbre.filsGauche))
fin

4.1.9 obtenirFilsGauche

fonction obtenirFilsGauche **(**arbre : ArbreHuffman**) :** ArbreHuffman
  ⌊**précondition(s)** non estFeuille(arbre)
debut
  retourner arbre.filsGauche
fin

4.1.10 obtenirFilsDroit

fonction obtenirFilsDroit **(**arbre : ArbreHuffman**) :** ArbreHuffman
  ⌊**précondition(s)** non estFeuille(arbre)

debut
  retourner arbre.filsDroit
fin

4.1.11 estPresent

fonction estPresent **(**arbre : ArbreHuffman, caractere : **Caractere) : Booleen**

debut
  si arbre.statistiquescaractere = caractere **alors**
    retourner Vrai
  sinon
    retourner estPresent(obtenirFilsDroit(arbre)) ou estPresent(obtenirFilsGauche(arbre))

  finsi
fin

4.1.12 copierArbreHuffman

fonction copierArbreHuffman **(**arbre : ArbreHuffman**) :** ArbreHuffman
  ⌊**précondition(s)** arbre ≠ NULL
  Déclaration stat : Statistiques, nouveauNoeud : ArbreHuffman
debut
  stat ← Statistiques.statistique(obtenirCaractere(arbre), obtenirPonderation(arbre))

  nouveauNoeud.filsGauche ← copierArbreHuffman(obtenirFilsGauche(arbre))

  nouveauNoeud.filsDroite ← copierArbreHuffman(obtenirFilsDroit(arbre))
  retourner nouveauNoeud
fin

4.2 Code Binaire

4.2.1 Déclaration du type

Type CodeBinaire **= Structure**
  bits : **Tableau**[1..MAX] **de** Bit
  taille : **Naturel**
finstructure

4.2.2 creerCodeBinaire

fonction creerCodeBinaire **(**taille : **Naturel) :** CodeBinaire
  Déclaration code : CodeBinaire, i : **Naturel**
debut
  code.taille ← taille
  pour i ←0 **à** 4 **faire**

code.bits[i] ← 0
finpour
retourner code
fin

4.2.3 remplirCodeBinaire

procédure remplirCodeBinaire **(E/S** code : CodeBinaire, **E** chaine : **Chaine de caracteres)**
⌊**précondition(s)** $longueur(chaine) \neq code.taille$
Déclaration i : **Naturel**
debut
pour i ←0 **à** code.taille **faire**
code.bits[i] ←
obtenirIemeCaractere(chaine,i)
finpour
fin

4.2.4 libererCodeBinaire

procédure libererCodeBinaire **(E/S** code : CodeBinaire**)**
debut
desallouer(code.bits)
code.taille ← 0
fin

4.2.5 estVide

fonction estVide **(**code : CodeBinaire**) : Booleen**
Déclaration taille : **Naturel**
debut
taille ← code.taille
retourner code = ""
fin

4.2.6 obtenirIemeBit

fonction obtenirIemeBit **(**code : CodeBinaire, i : **NaturelNonNul) :** Bit
⌊**précondition(s)** $i > 0$ et $i \leq longueur(code)$
debut
retourner code[i]
fin

4.2.7 concatener

fonction concatener **(**code1, code2: CodeBinaire**) :** CodeBinaire
Déclaration res : CodeBinaire, i : **Naturel**

debut
res.taille ← code1.taille + code2.taille
pour i ←0 **à** code1.taille **faire**
res.bits[i] ← code1.bits[i]
finpour
pour i ←0 **à** code2.taille **faire**
res.bits[i + code1.taille] ← code2.bits[i]
finpour
retourner res
fin

4.2.8 codeBinaireAStr

fonction codeBinaireAStr (code: CodeBinaire) **: Chaine de caracteres**
Déclaration str : **Chaine de caracteres**, i : **Naturel**
debut
pour i ←0 **à** code.taille **faire**
str ← str + code.bits[i]
finpour
retourner str
fin

4.2.9 obtenirCodeBinaire

fonction obtenirCodeBinaire (a : ArbreHuffman, car : **Caractere) : Chaine de caracteres**
⌊**précondition(s)** non ArbreHuffmanestVide(a).
debut
si ArbreHuffmanestFeuille(a).**alors**
retourner '0'
sinon
obtenirCodeBinaireR(a,car)
finsi
fin

4.2.10 obtenirCodeBinaireR

fonction obtenirCodeBinaireR (a : ArbreHuffman, car : **Caractere) : Chaine de caracteres**
⌊**précondition(s)** non(estVide(a))
Déclaration str,res : **Chaine de caracteres**, taille : **Naturel**
debut
si estFeuille(a) **alors**
retourner ""
finsi
si estPresent(obtenirFilsGauche(a),car) **alors**

```
        a ← obtenirFilsGauche(a)
        str ← obtenirCodeBinaireR(a,car)
        si str ≠ NULL alors
            res ← '0' + str
        finsi
        retourner res
    sinon
        a ← obtenirFilsDroit(a)
        str ← obtenirCodeBinaireR(a,car)
        si str ≠ NULL alors
            res ← '1' + str
        finsi
        retourner res
    finsi
fin
```

4.2.11 CbDeAH

```
fonction CbDeAH (a : ArbreHuffman, car : Caractere) : CodeBinaire
    Déclaration  res : CodeBinaire, str : Chaine de caracteres, taille :
                 Naturel
debut
    str ← obtenirCodeBinaire(a,car)
    taille ← longueur(str)
    res ← creerCodeBinaire(res,str)
    retourner res
fin
```

4.3 Statistiques

4.3.1 Déclaration du type

```
Type Statistiques = Structure
    caractere : Caractere
    ponderation : Naturel
finstructure
```

4.3.2 statistique

```
fonction statistique (c : Caractere, n : Naturel) : Statistiques
    Déclaration  stat : Statistiques
debut
    stat.caracteres ← c
    stat.ponderation ← n
    retourner stat
fin
```

4.3.3 obtenirPonderation

fonction obtenirPonderation (stat : Statistiques) **: Naturel**
debut
  retourner stat.ponderation
fin

4.3.4 obtenirCaractere

fonction obtenirCaractere (stat : Statistiques) **: Caractere**
debut
  retourner stat.caractere
fin

4.4 Octet

4.4.1 Déclaration du type

Type Octet **= Structure**
  Bit : **Tableau**[1..8] **de** Bit
finstructure

4.4.2 Variables Globales

positionDeDebut : **Entier**
actuelEstPlein : **Booleen**
bitRestant : Octet
indicePlusImportantDeBitRestant : **Entier**
  positionDeDebut = 1
actuelEstPlein = Faux
indicePlusImportantDeBitRestant = 9

4.4.3 obtenirBitRestants

fonction obtenirBitRestants () **:** Octet
debut
  retourner bitRestant
fin

4.4.4 obtenirIndicePlusImportantDeBitRestant

fonction obtenirIndicePlusImportantDeBitRestant () **: Entier**
debut
  retourner indicePlusImportantDeBitRestant
fin

4.4.5 incrementPositionDeDebut

fonction incrementPositionDeDebut () **:**
debut
  positionDeDebut ← positionDeDebut+1
fin
fonction obtenirPositionDeDebut () **:**
debut
  retourner positionDeDebut
fin

4.4.6 creerOctet

fonction creerOctet () **:** Octet
  Déclaration res : Octet, i : **Naturel**
debut
  positionDeDebut ← 0
  pour i ←1 **à** 8 **faire**
    res.bits[i] ← BITA0
  finpour
  retourner res
fin

4.4.7 fixerBit

procédure fixerBit (**E/S** octet : ˆOctet**E** pos : **Entier**, valeur : Bit)
  ⌊**précondition(s)** position*leq* 9
debut
  si position *leq* 9 **alors**
    octet→bits[position] ←
  finsi
fin

4.4.8 iemeBit

fonction iemeBit (octet : Octet, i : **Entier**) **:** Bit
debut
  retourner octet.bits[i]
fin

4.4.9 decaler

procédure decaler (**E/S** octet : ˆOctet)
  Déclaration i : **Naturel**
debut
  pour i ←1 **à** 8 **faire**
    octet→bits[i] ← bits[i+1]